

Assignment – 1 (CS 2123: Data Structures)

Name: Durga Jhansi Yerukala	ABC123: SCP203	Section: 004
-----------------------------	----------------	--------------

Table 1.1: Step Count & Big O

Best case step count	Worst-case Big O	Big O when $x = a[j]$
n	n	n

Table 1.2: Step Count OR Big O descriptions

Best-case step count	Line	Count	Description
<pre>int sequentialSearch(· · ·) { 1. int i; 2. for (i = 0; i < n && x != a[i]; i++); 3. if (i == n) return -1; 4. else return i; }</pre>	1	1	Variable is declared .
	2	1	Since best case is when $a[0] == x$, so the condition is checked once.
	3	1	The condition is still checked but it is false since $i = 0$;
	4	1	Since previous statement is checked and results in false, this statement is checked once , and still results false.
	Total step count =		4

Worst-case Big O	Line	Count	Description
<pre>int sequentialSearch(· · ·) { 1. int i; 2. for (i = 0; i < n && x != a[i]; i++); 3. if (i == n) return -1; 4. else return i; }</pre>	1	1	Variable is declared .
	2	n	Since worst case is when the element is not found in the array the loop traverse through array and compares x with every element .
	3	1	The condition is checked and results true.
	4	0	Since previous statement is true , this statement is not checked.
	Big O =		$O(n)$

Big O when $x = a[j]$	Line	Count	Description
<pre> int sequentialSearch(· · ·) { 1. int i; 2. for (i = 0; i < n && x != a[i]; i++); 3. if (i == n) return -1; 4. else return i; } </pre>	1	1	Variable is declared .
	2	$j+1$	Since the element is found at j index, the loop compares x with elements from $a[0]$ to $a[j]$ and stops when $x == a[j]$.
	3	1	The condition is still checked but it is false since $i = j+1$
	4	1	Since previous statement is checked and results in false, this statement is checked once , and still results false.
	Big O =		$O(n)$

Explanation for why Big(O) is n in the case where $x = a[j]$: the algorithm performs $j + 1$ comparisons, and since j can be as large as $n - 1$, the time complexity can grow with the size of the array. Therefore, even though the element may be found early sometimes, Big-O is $O(n)$ because Big-O doesn't consider constants.

- **No restrictions on word count.*
- **For worst case and $x = a[j]$, step counts are also required. But, in the end constant will be ignored i.e. Big O is n , $2n$, n^2 etc.*

Table 2.1: Big O representation

Big O representation for the squareMultiply	$O(n^3)$
---	----------

Table 2.2: Big O description

	Description
Big O representation for the squareMultiply	The algorithm has three nested loops each running from 0 to $n - 1$. For every pair of (i, j) , the inner loop performs n multiplications and additions, resulting in a total of $n \times n \times n = n^3$ operations.

**No restrictions on word count. In this case line by line step counts are not required. Only general description on how you computed the Big O.*

Table 3.1: Big O representation

Big O representation for the fastSquareMultiply	$O(n^3)$
---	----------

Table 3.2: Big O description

	Description
Big O representation for the fastSquareMultiply	The first two nested loops taking n^2 time. The following three nested loops perform the multiplication, which takes $n \times n \times n = n^3$ operations. Since n^3 dominates n^2 , the overall time complexity is $O(n^3)$.

**No restrictions on word count. In this case line by line step counts are not required. Only general description on how you computed the Big O.*